



Green University of Bangladesh

*Department of Computer Science and Engineering (CSE)
Semester: 12th (Fall, Year: 2026) / Bsc.in CSE (Day)*

Assignment Submission

Assignment Report

Course Title: Data Mining.

Course Code: CSE 435 / Section: 221_D5

Students Details

Name	ID
Khondokar Saim	221902353

Submission Date: 02/01/2026

Course Teacher's Name: Ayan Sarkar.

[For teachers use only: **Don't write anything inside this box**]

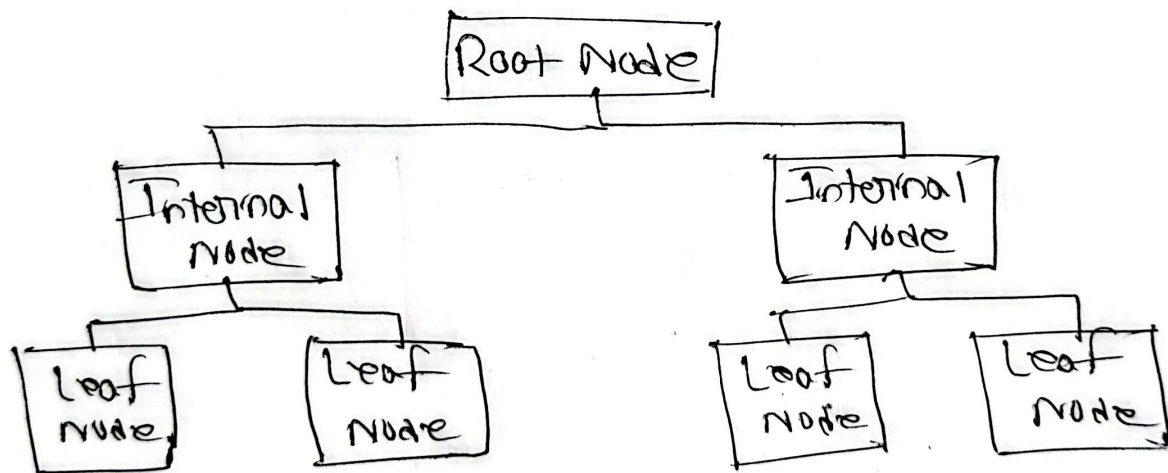
<u>Report Status</u>	
Marks:	Signature :
Comments:	Date:

Set 01: Decision Tree and 1 math with Entropy

What is Decision Tree

A Decision Tree helps us make decisions by showing different options and how they are related. It has a tree-like structure that starts with one main question called the Root Node which represents the entire dataset.

From there, the tree branches out into different possibilities based on features in the data.



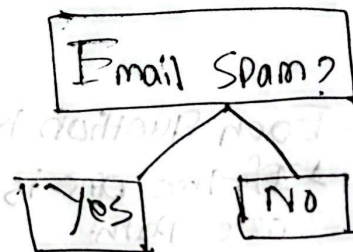
- * Root Node: Starting point representing the whole dataset.
- * Branches: Lines connecting nodes showing the flow from one decision to another.
- * Internal Nodes: Points where decisions are made based on data features.
- * Leaf Nodes: End points of the tree where the final decision / prediction is made.

Types of Decision Trees

There are mainly 2 types of decision trees based on the target Variable:

- 1) Classification Tree
- 2) Regression Tree.

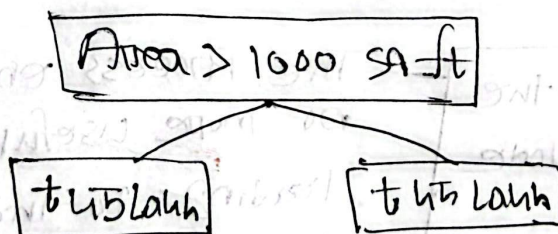
Classification Tree



Yes / No decision icons

Classification trees used for predicting categorical outcomes like spam/not spam. These trees split the data based on features to classify data into predefined categories.

Regression Tree



Regression trees are used for predicting continuous outcomes like predicting house prices. Instead of assigning categories, it provides numerical predictions based on the input features.

How Decision Trees Work?

Start with the Root Node

It begins with a main question at the root node which is derived from the dataset's features.

Ask Yes/No Questions

From the root, the tree asks a series of Yes/No questions to split the data into subsets based on specific attributes.

Branching based on Answers

Each question leads to different branches:
 → If the ans is Yes, the tree follows one path.
 → If the ans is No, the tree follows another path.

Continue Splitting

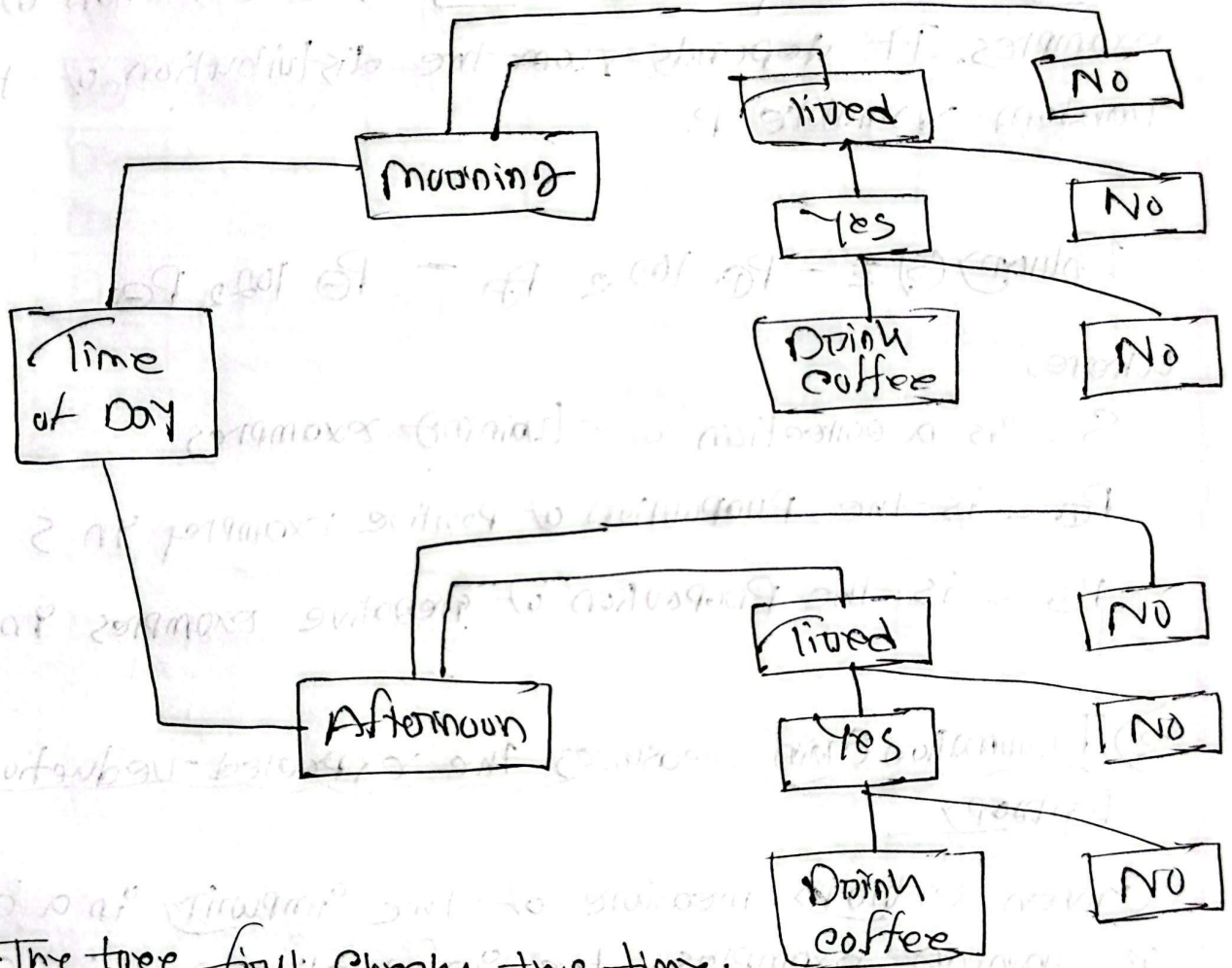
This branching continues through further decisions helps in reducing the data down step by step.

Reach the Leaf Node

The process ends when there are no more useful questions to ask leading to the leaf node where the final decision/prediction is made.

Example of Decision Trees Working Mechanisms

Let's look at a simple example to understand how it works. Imagine we need to decide whether to drink coffee based on the time of day and how tired we feel.



The tree first checks the time:

1) In the morning: It asks "Tired"?

- If yes, the tree suggests drinking coffee
- If no, it says no coffee is needed.

2) In the afternoon: It asks again "Tired"?

- If yes, it suggests drinking coffee
- If no, no coffee is needed.

Entropy and Information Gain in Decision Tree

1) Entropy measures Homogeneity

Entropy measures the impurity of a collection of examples. It depends from the distribution of the random variable P .

$$\text{Entropy}(S) \equiv -P_{+} \log_2 P_{+} - P_{-} \log_2 P_{-}$$

where,

S = is a collection of training examples

P_{+} = is the proportion of positive examples in S

P_{-} = is the proportion of negative examples in S

2) Information Gain measures the expected reduction in Entropy

Given entropy measure of the impurity in a collection of training examples, the information gain, is simply the expected reduction in entropy caused by partitioning the examples according to an attribute.

$$\text{Gain}(S, A) \equiv \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

MATH

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Q In this above datasets we want to calculate the Information Gain of the Wind attribute? $\text{Gain}(S, \text{Wind}) = ?$

Ans:

$\text{Values}(\text{Wind}) = \text{Weak}, \text{Strong}$

$S = [9+, 5-]$

$S_{\text{Weak}} \leftarrow [6+, 2-]$

$S_{\text{Strong}} \leftarrow [3+, 3-]$

Here, Now

$$\text{Entropy}([9+, 5-]) = -9/14 \log_2(9/14) - 5/14 \log_2(5/14) \\ \approx 0.940$$

$$\text{Entropy}([6+, 2-]) = -6/14 \log_2(6/14) - 2/14 \log_2(2/14) \\ \approx 0.811$$

$$\text{Entropy}([3+, 3-]) = -3/14 \log_2(3/14) - 3/14 \log_2(3/14) \\ \approx 1.00$$

Now for Grain,

$$\begin{aligned} \text{Grain}(S, \text{Wind}) &= \text{Entropy}(S) - \sum_{v \in \{\text{Sweat, Storm}\}} \frac{|S_v|}{|S|} \text{Entropy}(S_v) \\ &= \text{Entropy}(S) - (8/14) \text{Entropy}(S_{\text{Sweat}}) - (6/14) \text{Entropy}(S_{\text{Storm}}) \\ &= 0.940 - (8/14) 0.811 - (6/14) 1.00 \\ &= 0.048. \end{aligned}$$

Grain of two Cuid attribute is 0.048

Ans

Set 02: Concepts on Tree Splitting and Tree Pruning in Decision Tree with Example

Tree Splitting in Decision Tree

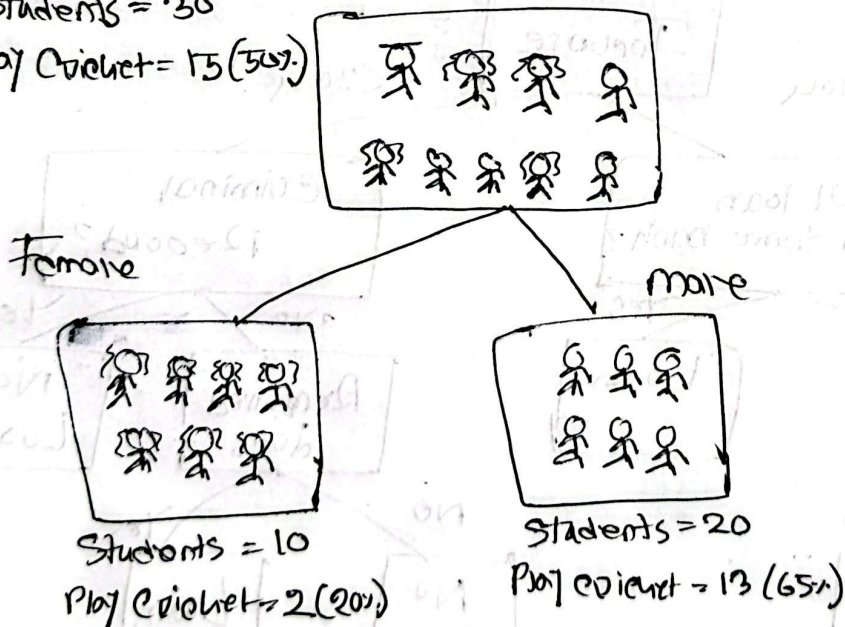
- In a decision tree, the process of splitting data at each node is important. The splitting criteria finds the best feature to split the data on.
- The process of partitioning the data set into subsets. Splits are formed on a particular variable.

Example

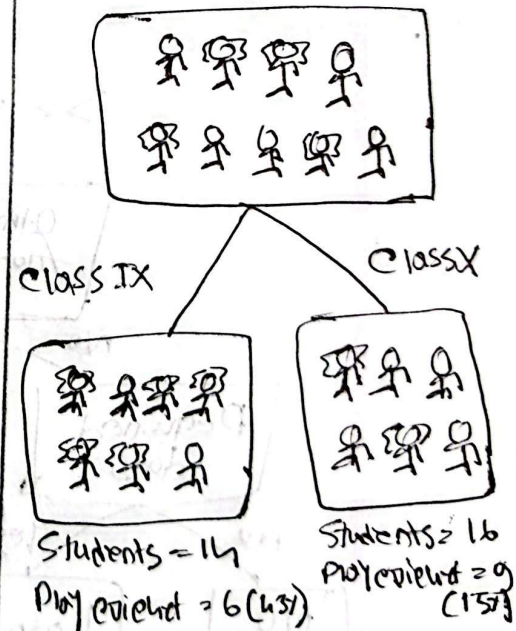
Split on Gender

Students = 30

Play Cricket = 15 (50%)



Split on Class



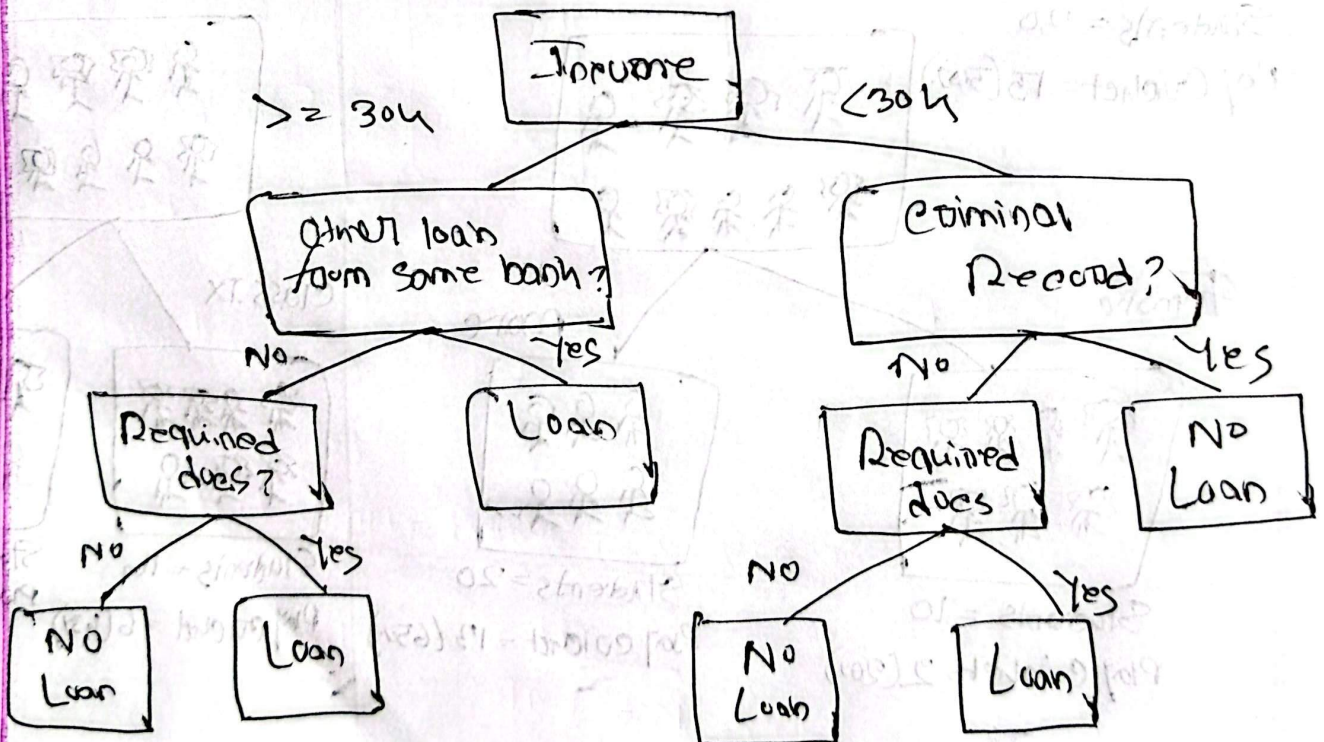
Splitting is done on various factors

Tree Pruning in decision Tree:

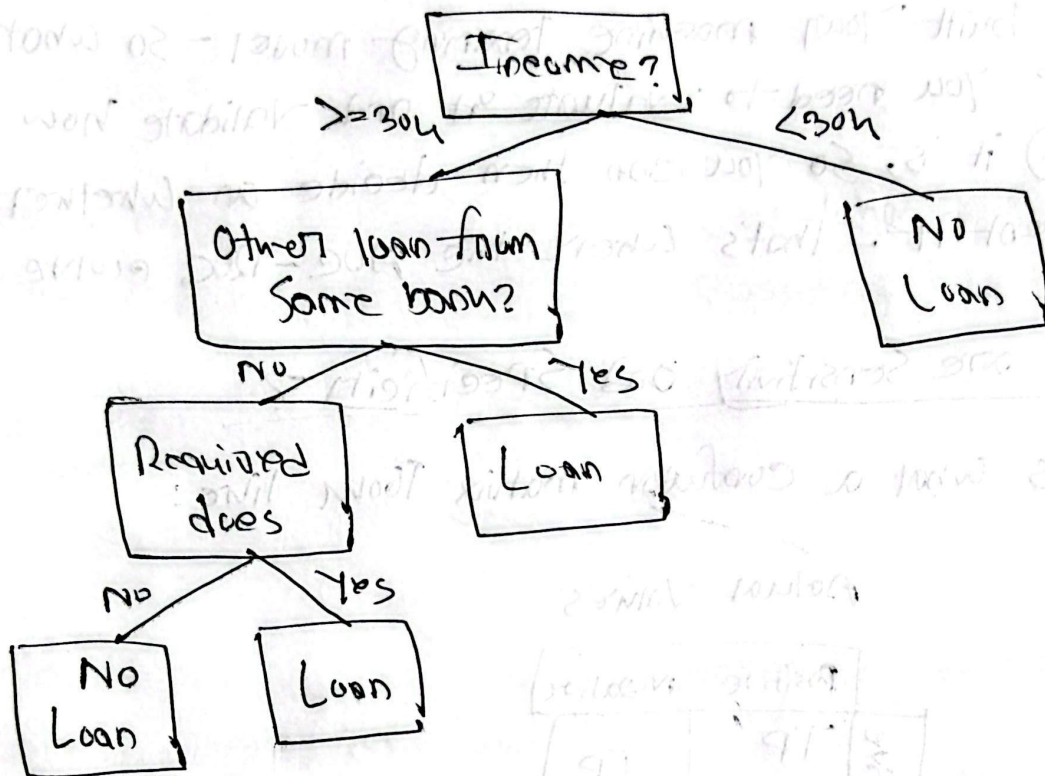
- Tree Pruning refers Shortening of branches of the tree. meaning,
- Pruning is the process of reducing the size of the tree by turning some branch nodes into leaf nodes, and removing the leaf nodes under the original branch.
- Pruning is useful because classification trees may fit the training data well but may do a poor job of classifying new values. A simpler tree often avoids over-fitting.

Example:

An UnPruned Decision Tree



A Pruned Decision Tree



As You can see a Pruned tree has less nodes and has less sparsity than a unpruned decision tree.

Set 03: ROC Curve, AUC

You've built your machine learning model - So what's next? You need to evaluate it and validate how good (or bad) it is, so you can then decide on whether to implement it. That's where the AUC-ROC curve comes in.

What are Sensitivity and Specificity?

This is what a confusion matrix looks like:

		Actual Values	
		Positive	Negative
Predicted Values	Positive	TP	FP
	Negative	FN	TN

From the confusion matrix, we can derive some important metrics.

① Sensitivity / True Positive Rate / Recall

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

Sensitivity tells us what proportion of the positive class got correctly classified.

② False Negative Rate

$$FNR = \frac{FN}{TP + FN}$$

False Negative Rate (FNR) tells us what proportion of the Positive class got incorrectly classified by the classifier.

③ Specificity / True Negative Rate:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Specificity tells us what proportion of the negative class got correctly classified.

④ False Positive Rate:

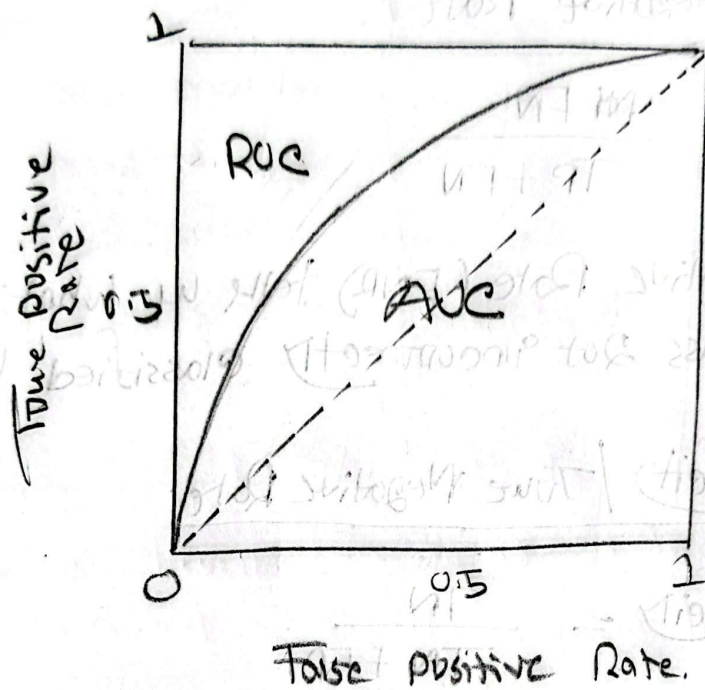
$$FPR = \frac{FP}{TN + FP} = 1 - \text{Specificity}$$

FPR tells us what proportion of the negative class got incorrectly classified by the classifier.

What is AUC - ROC curve:

AUC = Area Under the Curve

ROC = Receiver Operator Characteristic



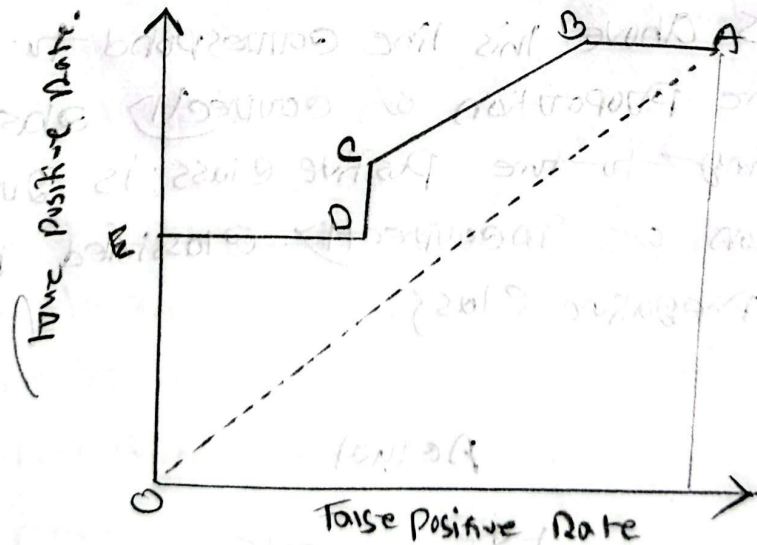
— The Receiver Operator Characteristic (ROC) curve is an evaluation metric for binary classification problem. It is a probability curve that plots the TPR against FPR at various threshold values and essentially separates the signal from the noise.

— The Area Under the Curve (AUC) is the measure of the ability of a classifier to distinguish between classes and is used as a summary of the ROC curve.

How does the AUC - ROC Curve Work?

- In an AUC - ROC curve, a higher X-axis value indicates a higher number of False Positives than True Negatives. While a higher Y-axis value indicates a higher number of True Positives than False Negatives. So the choice of the threshold depends on the ability to balance True Positives and False Negatives naturally.

Let's dig a bit deeper and understand how our ROC curve would live for different threshold values and how the Specificity and Sensitivity would vary.



We can try and understand this graph by generating a confusion matrix for each point corresponding to a threshold and talk about the performance of our classifier.

		Actual	
		+	-
Predicted	+	5	5
	-	0	0

Point A is where the Sensitivity is the highest and Specificity the lowest. This means all the positive class points are classified correctly and all the negative class points are classified incorrectly.

In fact, any point on the blue line corresponds to a situation where the true Positive Rate is equal to False Positive Rate.

All points above this line correspond to the situation where the proportion of correctly classified points belonging to the positive class is greater than the proportion of incorrectly classified points belonging to the negative class.

		Actual	
		+	-
Predicted	+	5	4
	-	0	4

Although Point D has the same Sensitivity as Point A, it has a higher Specificity. meaning the number of incorrectly Negative class points is lower than the previous threshold. This indicates that this threshold is better than the previous one

Point C

	Actual	
Predicted	+ve	-ve
	4	2
	1	3

$$\text{TPR/Sensitivity} = 0.8$$

$$\text{FPR} = 0.4$$

$$\text{Specificity} = 1 - \text{FPR} = 0.6$$

Point D

	Actual	
Predicted	+ve	-ve
	3	2
	2	3

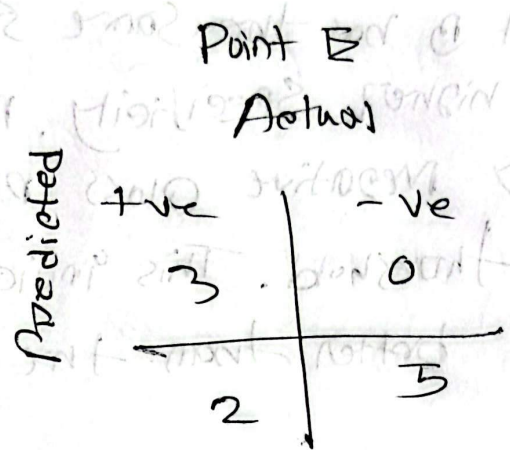
$$\text{TPR/Sensitivity} = 0.6$$

$$\text{FPR} = 0.4$$

$$\text{Specificity} = 1 - \text{FPR} = 0.6$$

Between Points C and D, the Sensitivity at Point C is higher than Point D for the same Specificity. This means, for the same number of incorrectly classified Negative class points, the classifier predicted a higher number of positive class points. Therefore, the threshold at Point C is better than Point D

Now, depending on how many incorrectly classified points we want to tolerate for our classifier, we could choose between Point C or D to predict whether you can defeat me in PUBG / not.



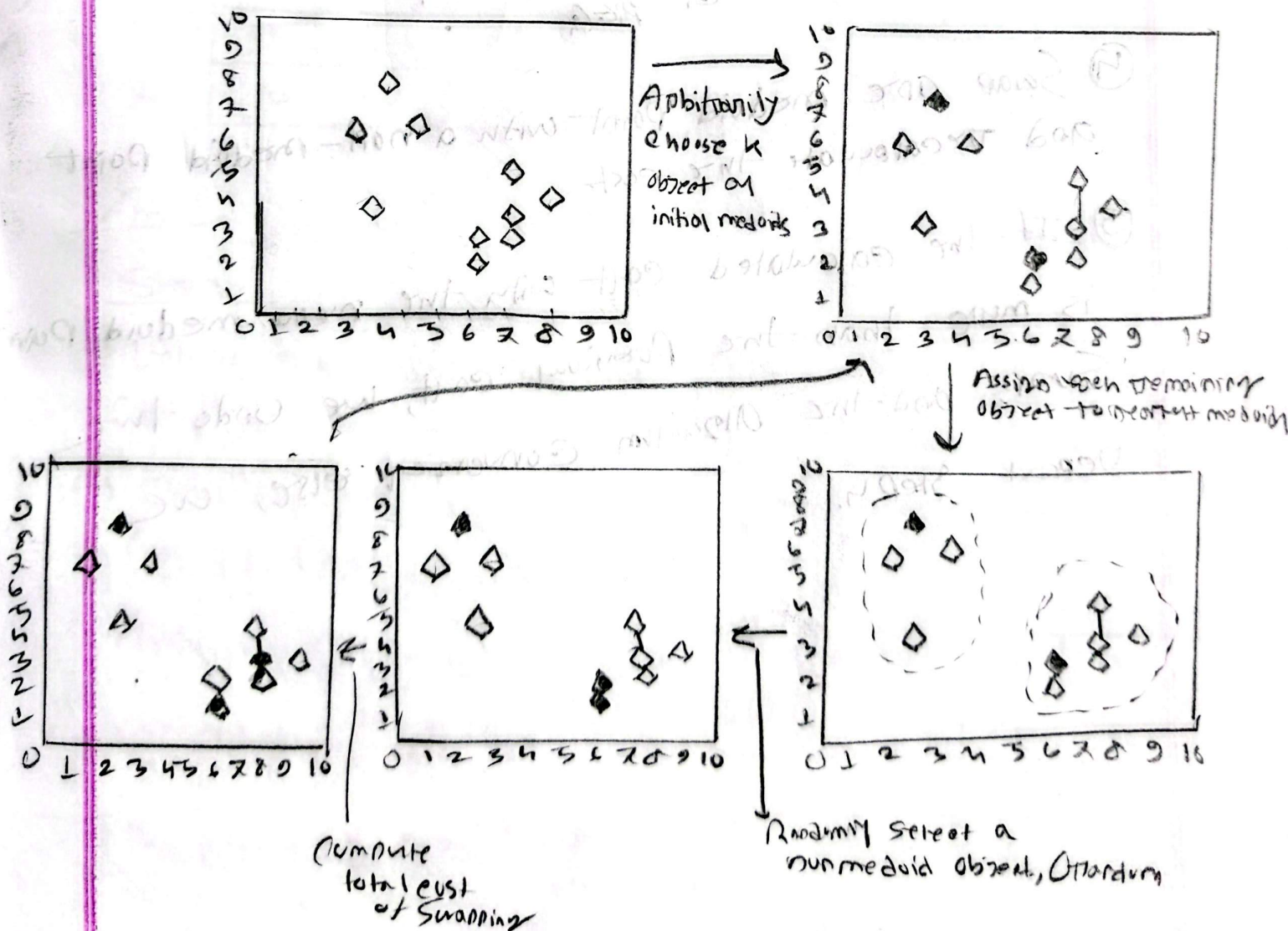
$TPR / \text{sensitivity} = 0.6$
 $FPR = 0$
 $\text{Specificity} = 1 - FPR = 1$

Point B is where the Specificity becomes highest, meaning the model classifies no False Positives.

The model can correctly classify all the Negative class points? We would choose this Point if our problem was to give perfect song recommendation to our users.

Set on: Cluster Analysis using k-medoids

- Partitioning Around Medoids or the k-medoids algorithm is a Partitional clustering algorithm which is slightly modified from the k-means algorithm.
- In k-means algorithm, they choose means or the Centroids but in the k-medoids, data points are chosen to be the medoids.



① Initially Select k -random points as the medoids from the given n data points $\checkmark$ - the data set.

② Associate each data point to the closest medoid by using any of the most common distance metrics.

③ Calculate the cost as the total sum of the distances (also called dissimilarities) $\checkmark$ the data points from the assigned medoid.

$$C = \sum_{c_i} \sum_{P_i \in C_i} |P_i - c_i|$$

④ Swap one medoid point with a non-medoid point and recalculate the cost

⑤ If the calculated cost with the new medoid P_{new} is more than the previous cost, we undo the swap, and the algorithm converges else, we repeat step 4.

MATH:

Point	X	Y
1	1	4
2	5	1
3	5	2
4	5	4
5	10	4
6	25	4
7	25	6
8	25	7
9	25	8
10	29	7

Question

Apply k-medoid clustering algorithm to form the clusters. Use Euclidean distance to find the distance between data point and medoid.

Ans

Step 1:

→ Let us choose that (1,4) and (10,4) are the medoids.

→ $c_1 = (1,4)$ and $c_2 = (10,4)$

→ (m_1, y_1) and (m_2, y_2) are data points

→ Euclidean Dist = $\sqrt{(m_2 - m_1)^2 + (y_2 - y_1)^2}$

→ $ED_{14} [(1,4), (1,4)] = \sqrt{(1-1)^2 + (4-4)^2} = 0$

→ $ED_{14} [(1,4), (5,1)] = \sqrt{(5-1)^2 + (1-4)^2} = 5$

Point	X	Y	c1	c2	Cluster
1	1	4	0	9	c1
2	5	1	5	5.83	c1
3	5	2	4.47	5.39	c1
4	5	4	4	5	c1
5	10	4	9	0	c2
6	25	4	24	13	c2
7	25	6	24.8	15.13	c2
8	25	7	24.19	15.30	c2
9	25	8	24.33	15.32	c2
10	29	7	28.14	19.24	c2

Cluster one:

$$C_1 = \{(1,4), (5,1), (5,2), (5,4)\} \text{ and}$$

$$C_2 = \{(10,4), (25,4), (25,6), (25,2), (25,8), (29,2)\}$$

Now calculating the cost which is nothing but the sum of distance of each non-medoid point from the medoid of the cluster it belongs to.
Here medoid

Calculate the total cost =

$$\text{Cost}(C, m) = \sum_i |e_i - m|$$

$$\begin{aligned} \text{Total cost} &\rightarrow \text{cost}((1,4), (5,1)) + \text{cost}((1,4), (5,2)) + \\ &\quad \text{cost}((1,4), (5,4)) + \text{cost}((10,4), (25,4)) + \\ &\quad \text{cost}((10,4), (25,6)) + \text{cost}((10,4), (25,2)) + \\ &\quad \text{cost}((10,4), (25,8)) + \text{cost}((10,4), (29,2)) \end{aligned}$$

$$\begin{aligned} \therefore \text{Total cost} &= \{12-5| + |4-1|\} + \{12-5| + |4-2|\} + \{12-5| + |4-4|\} \\ &\quad + \{10-25| + |4-4|\} + \{10-25| + |4-6|\} + \{10-25| + |4-2|\} + \{10-25| + |4-8|\} + \\ &\quad \{10-29| + |4-2|\} \end{aligned}$$

$$= 4 + 3 + 4 + 2 + 4 + 0 + 15 + 0 + 15 + 2 + 15 + 3 + 15 + 4 + 10 + 3$$

$$= 108$$

Step-2

$$c_1 = (5, 4)$$

$$c_2 = (25, 7)$$

Euclidean Dist

$$= \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$E_{\text{dist}}[(1, 4), (5, 4)] = \sqrt{(5-1)^2 + (4-4)^2}$$

$$= 4$$

$$E_{\text{dist}}[(1, 4), (25, 7)] = \sqrt{(25-1)^2 + (7-4)^2}$$

$$= 24.19$$

Now,

$$c_1 = \{(1, 4), (5, 1), (5, 2), (5, 4), (10, 4)\} \text{ and}$$

$$c_2 = \{(25, 4), (25, 6), (25, 7), (25, 8), (29, 7)\}$$

Calculate the total cost

$$\text{Cost}(c, n) = \sum_i |c_i - n_i|$$

$$\begin{aligned} \text{Total cost} = & \text{cost}((5, 4), (1, 4)) + \text{cost}((5, 4), (5, 1)) + \\ & \text{cost}((5, 4), (5, 2)) + \text{cost}((5, 4), (10, 4)) + \\ & \text{cost}((25, 7), (25, 4)) + \text{cost}((25, 7), (25, 6)) + \\ & \text{cost}((25, 7), (25, 8)) + \text{cost}((25, 7), (29, 7)) \end{aligned}$$

$$\therefore \text{Total cost} = 23$$

Point	X	Y	c1	c2	cluster
1	1	4	4	24.19	c1
2	5	1	3	20.8	c1
3	5	2	2	20.62	c1
4	5	4	0	20.22	c1
5	10	4	5	15.30	c1
6	25	4	20	3	c2
7	25	6	20.16	1	c2
8	25	7	20.22	0	c2
9	25	8	20.40	1	c2
10	29	7	24.19	4	c2

→ Swapping cost = current Total cost - Previous Total cost

$$= 223 - 108$$

$$= 115 < 0$$

If Swapping cost > 0, we stop the algorithm here and cluster formed in step 2 is the final Answer

$$\{(1,2), (1,3), (1,4), (1,5)\} = 10$$

$$\{(2,3), (2,4), (2,5), (3,4), (3,5)\} = 5$$

∴ Total cost = 10 + 5 = 15

∴ Final Answer is 15

$$\begin{aligned} &+ ((1,2), (1,3)) + 100 + ((1,4), (1,5)) + 100 \\ &+ ((1,4), (1,5)) + 100 + ((2,3), (2,4)) + 100 \\ &+ ((2,3), (2,4)) + 100 + ((2,5), (3,4)) + 100 \\ &+ ((2,5), (3,4)) + 100 + ((3,4), (3,5)) + 100 \end{aligned}$$

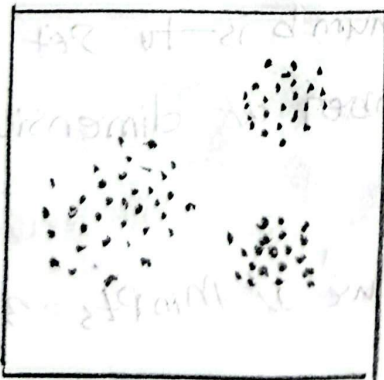
$$25 = 1200 \text{ total}$$

Set 05: Density-based Clustering (DBSCAN)

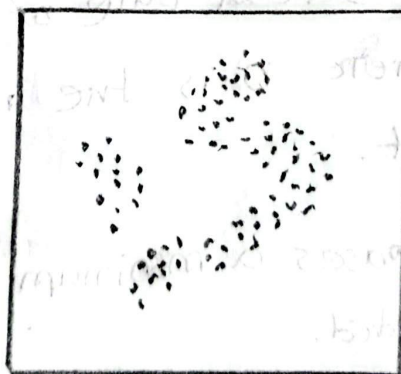
DBSCAN is a density based clustering algorithm that groups data points that are closely packed together and marks outliers as noise based on their density in the feature space. It identifies clusters as dense regions in the data space separated by areas of lower density. Unlike k-means or hierarchical clustering which assumes clusters are compact and spherical, DBSCAN performs well in handling real world data irregularities such as:

- 1) Arbitrary-Shaped Clusters: Clusters can take any shape not just circular or convex.
- 2) Noise and Outliers: It effectively identifies and handles noise points without assigning them to any cluster.

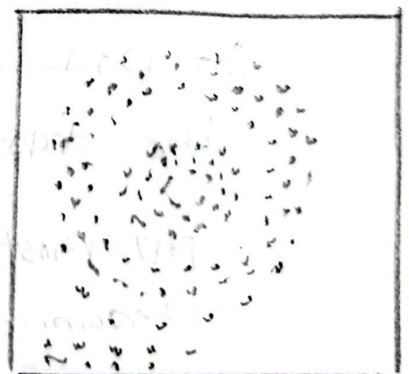
Data Base 1



Data Base 2



Data Base 3



The figure above shows a data set which clustering algorithms like k-means or hierarchical clustering handle poorly, while DBSCAN manages arbitrary-shaped clusters and noise handling.

Key Parameters in DBSCAN

1) eps: This defines the radius of the neighborhood around a data point. If the distance between two points is less than or equal to ϵ they are considered neighbors. A common method to determine ϵ is by analyzing the k -distance graph. Choosing the right ϵ is important.

- If ϵ is too small most points will be classified as noise.
- If ϵ is too large clusters may merge and the algorithm may fail to distinguish between them.

2) minPts: This is the minimum number of points required within the ϵ radius to form a dense region. A general rule of thumb is to set $\text{minPts} \geq D+1$ where D is the number of dimensions in the dataset.

For most cases a minimum value of $\text{minPts} = 3$ is recommended.

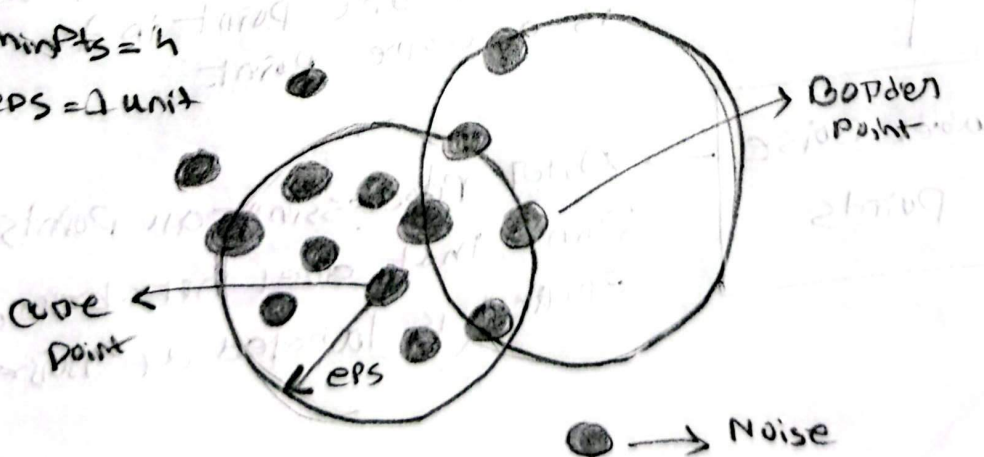
How does DBSCAN work?

DBSCAN works by categorizing data points into 3 types -

- 1) Core Points which have a sufficient number of neighbours within a specified radius (epsilon)
- 2) Border points which are near core points but lack enough neighbours to be core points themselves.
- 3) Noise points which do not belong to any cluster

By iteratively expanding clusters from core points and connecting density-reachable points, DBSCAN forms clusters without relying on rigid assumptions about their shape or size.

$\text{minPts} = 4$
 $\text{eps} = 1 \text{ unit}$



Steps in the DBSCAN Algorithm

Identify core points

For each point in the dataset count the number of points within its ϵ neighborhood. If the count meets or exceeds minPts mark the point as a core point.

Form clusters

For each core point that is not already assigned to a cluster create a new cluster. Recursively find all density connected points.

Density Connectivity

Two points a and b are density connected if there exists a chain of points where each point is within the ϵ radius of the next and at least one point in the chain is a core point.

Label Noise points

After processing all points any point that does not belong to a cluster is labeled as 'noise'.

MATH:Data Points:

$P_1: (3, 2), P_2: (4, 6), P_3: (5, 5), P_4: (6, 4), P_5: (7, 3),$
 $P_6: (6, 2), P_7: (7, 2), P_8: (8, 4), P_9: (3, 3), P_{10}: (2, 4),$
 $P_{11}: (3, 5), P_{12}: (2, 4).$

Apply the DBSCAN algorithm to the given data points and create the clusters with $\text{minPts} = 4$ and $\text{epsilon}(\epsilon) = 1.9$.

Ans

We know that Euclidean distance for distance between each points -

$$\text{Distance}(A(x_1, y_1), B(x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$\text{minPts} = 4$ and $\text{epsilon}(\epsilon) = 1.9$

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12
P1	0											
P2	1.41	0										
P3	2.83	1.41	0									
P4	4.24	2.83	1.41	0								
P5	5.66	4.24	2.83	1.41	0							
P6	5.83	4.47	3.16	2.00	1.41	0						
P7	6.40	5.00	3.61	2.24	3.00	1.00	0					
P8	5.83	4.47	3.16	2.00	1.41	2.83	2.24	0				
P9	4.00	3.16	2.83	3.16	4.00	3.16	4.12	5.10	0			
P10	1.41	2.00	3.16	4.47	5.83	5.66	6.40	6.32	3.16	0		
P11	2.00	1.41	2.00	3.16	4.47	4.24	5.60	5.10	2.00	1.41	0	
P12	3.16	2.83	3.16	4.00	5.10	4.47	5.32	6.00	1.41	2.00	1.41	0

Now,

$P_1 : P_2, P_{10}$

$P_2 : P_1, P_3, P_{11}$

$P_3 : P_2, P_4$

$P_4 : P_3, P_5$

$P_5 : P_4, P_6, P_7, P_8$

$P_6 : P_5, P_7$

$P_7 : P_5, P_6$

$P_8 : P_5$

$P_9 : P_{12}$

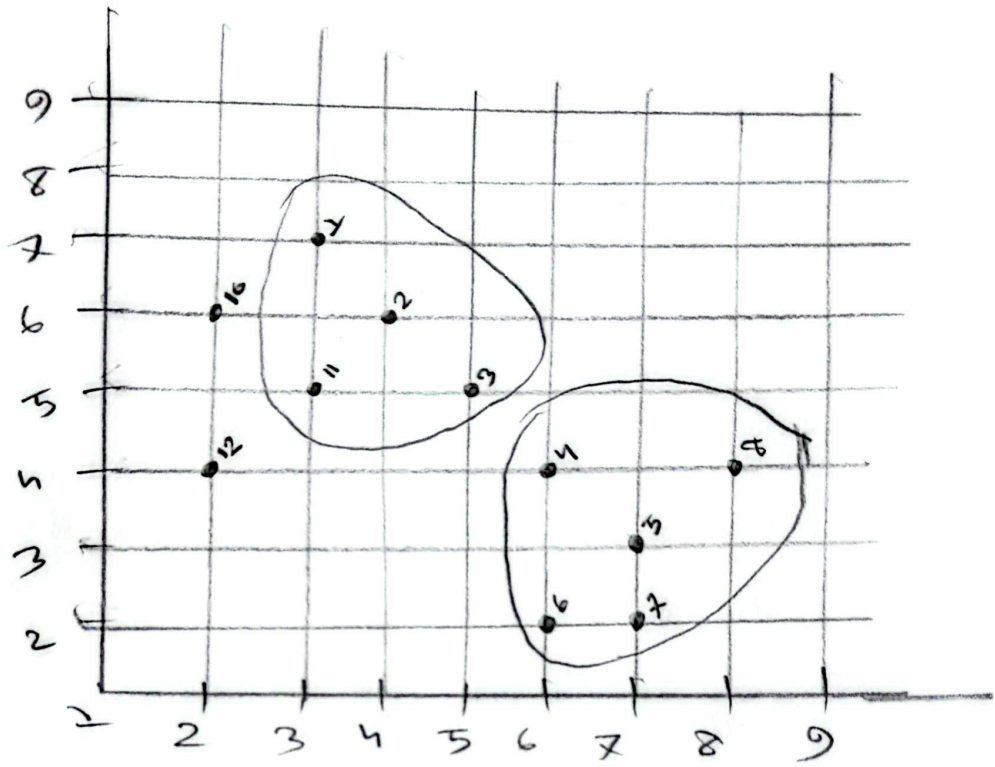
$P_{10} : P_1, P_{11}$

$P_{11} : P_2, P_{10}, P_{12}$

$P_{12} : P_9, P_{11}$

$\text{MinPts} = 4$ and $\text{Epsilon}(\epsilon) = 1.9$

Point	Status	
P_1	Noise	Border
P_2	Core	
P_3	Noise	Border
P_4	Noise	
P_5	Core	
P_6	Noise	Border
P_7	Noise	Border
P_8	Noise	Border
P_9	Noise	
P_{10}	Noise	Border
P_{11}	Core	
P_{12}	Noise	Border



Ans